Rapport de projet – Programmation Impérative (PR3)

1. Résumé

Ce projet consiste à concevoir un routeur logiciel capable de déterminer l'interface de sortie d'un paquet à partir d'une table de routage. La table est stockée sous forme de liste chaînée et exploitée selon la règle du masque le plus long. Afin d'optimiser les performances, un cache est ajouté et géré selon différentes politiques (FIFO, LRU,). Le cache peut être implémenté à l'aide d'une liste chaînée ou d'un arbre préfixe. Le programme traite des fichiers d'entrée et fournit des résultats ainsi que des statistiques d'exécution.

2. Introduction

Le routeur est un organe vital du réseau dont la fonction est de diriger chaque paquet entrant vers la bonne interface de sortie en consultant une table de routage .

Ce routeur peut être :

- **Simple**: donne l'interface de sortie en comptant sur seulement la table de routage.
- **Avec cache**: introduisant une structure de cache qui permet d'accélérer le process de la recherche de la bonne interface.

Le cache ne peut pas cependant contenir toutes les informations de la table de routage et garantir la rapidité en même temps c'est pourquoi il doit être de taille petite et suit une politique de suppression pour gérer efficacement les données qu'il les garde.
De plus le cache va être implémenter en deux structures de données afin de comparer l'efficacité de ces deux structures.

Le plan du document est comme suit :

- ✓ **Présentation des principaux choix réalisés**
- ✓ **Présentation des principaux algorithmes et types de données**
- ✓ **Démarche adoptée pour tester le programme**
- ✓ **Difficultés rencontrées et solutions adoptées**
- ✓ **Organisation de l'équipe**

- ✓ **Bilan technique et perspectives**

3. Architecture de l'application en modules

L'application est structurée en modules cohérents, chacun répondant à une responsabilité précise pour garantir la modularité et l'efficacité du routeur:

- **Module IP** : Ce module central définit les types fondamentaux T_Octet et T_Adresse_IP (entier modulaire sur 32 bits). Il fournit les outils de manipulation binaire nécessaires au routage, comme l'initialisation d'IP à partir d'entiers, le calcul de la longueur des chaînes, l'écriture formatée dans les fichiers de résultats, et la procédure critique Ajoute_bit_paquet_a_destination_masque pour la construction de routes discriminantes.
- **Module TR (Table de Routage)** : Il implémente le stockage de la table de routage complète via une liste chaînée. Ce module permet non seulement l'enregistrement et la suppression de routes, mais offre surtout des fonctions de recherche avancées comme Interface_f (pour trouver l'interface associée à un paquet) et Nombre_Destination_corresp (pour chercher le nombre de destination correspondant à un préfixe de paquet).
- **Module ARBRE** : Il implémente la structure du cache sous forme d'**arbre préfixe**. Cette structure optimise la recherche en parcourant les bits de l'adresse IP (0 ou 1) pour discriminer les informations de routage de manière efficace.
- **Module LC** : Il définit la structure de données du cache lorsqu'il est représenté par une liste chaînée. Il gère l'insertion, la recherche rapide de l'interface associée à un paquet et la suppression de routes spécifiques lors de l'éviction.
- **Module LP (Liste de Priorité)** : Ce module gère l'ordonnancement des données du cache selon l'ordre d'entrée ou de fréquence. Il est indispensable pour l'application des politiques d'éviction (FIFO, LRU) en identifiant l'élément à supprimer lorsque le cache atteint sa taille maximale.
- **Module CACHE** : Véritable interface d'abstraction, ce module encapsule les trois modules précédents (LC, ARBRE et LP). Il permet au programme principal d'utiliser le cache de manière uniforme, quelle que soit la structure de stockage choisie (Arbre ou Liste) et la politique de suppression active, garantissant ainsi la cohérence du routage.

4. PRÉSENTATION DES PRINCIPAUX CHOIX RÉALISÉS

- **Encapsulation** : L'utilisation de types limited private pour interdire la copie superficielle de la structure de données et ceci car les modules contiennent des pointeurs vers de la mémoire dynamique donc une copie simple risquerait de créer des alias dangereux où deux variables pointeraient vers la même zone mémoire, provoquant des erreurs lors de la libération
- **Gestion dynamique** : Le choix de la mémoire dynamique pour l'arbre et les listes permet de gérer des tables de tailles variables, avec une libération rigoureuse.
- **Distinction entre Table de Routage (TR) et Cache (LC) :** Bien que la table de routage et une version du cache utilisent toutes deux des listes chaînées, nous avons choisi de créer un module LC distinct du module TR. Ce choix est motivé par deux raisons majeures :
    - Spécificité fonctionnelle. En effet la fonction de recherche dans le cache (Chercher_Interface_LC) diffère de celle de la table de routage (Interface_f). Le cache contient des routes déjà optimisées (masques discriminants), ce qui simplifie la logique de recherche par rapport à la table complète.
    - Clarté architecturale car séparer ces modules permet une structuration plus propre du code (principe de responsabilité unique), évitant de surcharger le module TR avec des fonctions spécifiques à la gestion temporaire du cache.
- **Centralisation de la politique de gestion du cache via le module LP :** Plutôt que d'intégrer la logique de remplacement (FIFO, LRU) directement au sein des structures ARBRE ou LC, nous avons opté pour le module LP (Liste de Priorité) car :

  Premièrement, gérer la priorité directement dans un arbre préfixe est complexe (Il faut faire des compteurs dans chaque nœud etc …).

  Deuxièmement le découplage du problème : le module CACHE interroge LP pour savoir quel élément supprimer, puis demande au module ARBRE ou LC d'exécuter la suppression physique. Cela rend le système beaucoup plus facile à faire.

5. ALGORITHMES ET TYPES DE DONNÉES PRINCIPAUX

5.1 Algorithmes de routage et de mise à jour

5.1.1 Recherche du masque le plus long (Interface_f)

L'algorithme de recherche dans la table de routage complète repose sur le principe de la correspondance la plus spécifique. Dans la fonction Interface_f , où nous implémentons cet algorithme, nous parcourons l'intégralité de la table de routage et pour chaque couple (destination, masque), nous vérifions si le paquet lui correspond. Si plusieurs couples sont compatibles, nous mémorisons celui possédant le masque le plus long (déterminé par la fonction longueur_adresse_IP). Cela garantit que le paquet est envoyé vers l'interface la plus précise disponible.

5.1.2 Algorithme de cohérence et mise à jour du Cache

L'algorithme Mettre_A_Jour_CACHE est le cœur du routeur avec cache. Son rôle est d'insérer une route dans le cache tout en garantissant qu'elle ne provoquera pas d'erreurs de routage ultérieures (cohérence du cache).

- Calcul du masque discriminant : Au lieu d'insérer la route exacte de la table de routage, nous utilisons une boucle while qui affine la route bit par bit via Ajoute_bit_paquet_a_destination_masque qui ajoute bit par bit les bits du paquet à la destination en ajoutant des bits au masque.
- Condition d'arrêt : La boucle s'arrête dès que Nombre_Destination_corresp indique qu'une seule route de la table de routage correspond à ce nouveau préfixe. Cela permet d'avoir des entrées de cache plus générales, augmentant ainsi le taux de succès (hit rate).
- Gestion de la suppression : Avant l'insertion, le module interroge la liste de priorité (LP). Si le cache est plein, l'élément le plus ancien ou le moins utilisé est supprimé simultanément de la LP et de la structure de stockage (ARBRE ou LC).

5.2 Types de données

5.2.1 La Liste Chaînée (Module LC)

Pour la table de routage et l'une des versions du cache, nous utilisons une liste chaînée simple. Chaque cellule contient l'adresse destination, le masque, l'interface et un pointeur vers l'élément suivant. Cette structure est flexible pour les insertions, mais nécessite un parcours linéaire pour la recherche.

5.2.2 L'Arbre Préfixe (Module ARBRE)

L'arbre préfixe (ou *trie*) est la structure la plus optimisée pour ce projet. Elle organise le cache de manière hiérarchique en suivant la représentation binaire des adresses IP.
Elle réponds à ces critères :

- ✓ Chaque niveau de l'arbre correspond à un bit spécifique de l'adresse IP, en commençant par le bit de poids fort (niveau 0, bit 31). Pour chaque bit, deux directions sont possibles : le fils gauche représente le bit 0 et le fils droit le bit 1.
- ✓ Un nœud ne contient une information de routage que si le chemin parcouru depuis la racine (qui est le préfixe) est suffisant pour discriminer une route par rapport aux autres règles de la table de routage. On le marque dans ce cas par le booléen Est_Route et il stocke l'interface de sortie.
- ✓ Le masque est intrinsèquement lié à la structure (n'a pas besoin d'être stocké explicitement). En effet La longueur du masque correspond exactement au niveau de profondeur où la donnée est insérée.

  Exp : Une destination insérée au niveau 8 de l'arbre possède implicitement un masque de 8 bits (255.0.0.0).

6. DÉMARCHE DE TEST

La validation du projet a suivi une approche incrémentale, une vérification qui va du bas niveau jusqu'à la validation fonctionnelle globale du routeur.

6.1 Tests unitaires des modules de base

Avant l'intégration, chaque module (**Module TR / Module LP / Module Cache)** a été validé par un programme de test dédié pour garantir la robustesse des fondations du système :

6.2 Analyse de la gestion mémoire

Conformément aux critères de notation, une attention particulière a été portée à la robustesse mémoire: les programmes ont été passés sous Valgrind pour s'assurer de l'absence de fuites mémoire (memory leaks), validant ainsi l'efficacité de nos procédures Detruire dans les différents modules .

6.3 Comparaison systématique

Comparaison systématique des fichiers resultats.txt entre les versions routeur_LL et routeur_LA pour s'assurer de l'identité des comportements.

7. DIFFICULTÉS RENCONTRÉES ET SOLUTIONS

7.1 Complexité du raffinage d'un système d'envergure

La difficulté majeure a été de mener à bien le raffinage complet d'un programme de cette taille. Décomposer le problème principal en sous-problèmes gérables tout en anticipant les interactions entre les modules .

Solution : Nous avons laissé les actions complexes faites dans les modules pour ne pas les écrire dans le raffinage et se concentrer à une bonne et claire structure de raffinement .

7.2 Compréhension et implémentation du préfixe discriminant

L'une des parties les plus ardues fut la compréhension et l'implémentation du principe de cohérence du cache via l'ajout de bits.

Le défi était de s'assurer que la route insérée dans le cache ne soit pas trop générale (ce qui causerait des erreurs de routage) ni trop spécifique (ce qui rendrait le cache inefficace).

Solution: Nous avons dû développer une logique de boucle utilisant la fonction Nombre_Destination_corresp. Cette boucle ajoute bit par bit les informations du paquet à la destination et met à jour le masque jusqu'à ce que le couple devienne **discriminant**, c'est-à-dire qu'il ne corresponde plus qu'à une seule interface unique dans la table de routage.

8. ORGANISATION DE L’ÉQUIPE

ABID Mohamed :

- Conception des raffinages.
- Développement et tests unitaires des modules IP, TR, LP, LC, ARBRE et CACHE.
- Test des programmes finaux (routeur_LL/routeur_LA) et rédaction du rapport.

LUDWIGS Cédric :

- Développement des programmes principaux routeur_LL et routeur_LA (gestion CLI et fichiers E/S).
- Rédaction du manuel utilisateur.

9. BILAN TECHNIQUE ET PERSPECTIVES

Le système est opérationnel et respecte les contraintes de performance imposées.

Une évolution possible serait d’améliorer les procédures du module Arbre et l’implémentation également de la **table de routage sous forme d'arbre.**